ALKIS / AFIS / ATKIS — Auto-Datenvalidation

Konsolidiertes Konzept: Regelbasierte Pipeline (RF → Decision Tree → SkopeRules → SQL) erweitert um eine vorgeschaltete Self-Supervised-Learning-Schicht mit CLIP-Text-Embeddings. Fokus: Tabellendaten, Freitext und Geometrie-Features. Keine Bilddaten / Orthofotos.

> **Ziel:** ML-Modelle extrahieren Muster aus Geodaten. Das System speichert Ergebnisse als menschenlesbare SQL/Python-Regeln. Kein Blackbox-ML im Betrieb. Jede Regel muss von einem Domain-Experten bestätigt werden.
>
> **Kernprinzip:** ML wird nur zur *Regelableitung* genutzt — nicht zur Laufzeit-Validierung. Jede gefundene Regel wird als SQL CHECK oder Python-Callable gespeichert und im Human-in-the-Loop-Prozess freigegeben.

Architektur-Überblick

Phase	Input	Methode	Output	source
0 NEU	Alle Tabellen + Freitext + Geometrie	SSL + CLIP → 64D Latent + 2D UMAP	Embeddings, Cluster, Outlier	ssl_embedding
1	Stratifiziertes Sample (per Cluster)	Profiling	min/max/cat-Regeln	profiling
2	Top-Features (RF) + optional Cluster-Label als Target	Decision Tree, SkopeRules, FP-Growth, Lasso	IF-THEN SQL-Regeln	ml
2.5 NEU	Cross-Table Embeddings	Cluster-Mapping zwischen Tabellen	FK-Integritätsregeln	ssl_cross_table
2.6 NEU	Freitext vs. strukturierte Spalten	CLIP Cosine-Similarity	Semantik-Mismatch-Regeln	ssl_clip_text
3	Manuelle Entity-Links	User-definiert	FK-Regeln	manual
4	Alle proposed Regeln + 2D-UMAP-Plot	Human-in-the-Loop mit visuellem Kontext	active / rejected	—

Phase 0 (NEU) — Repräsentations-Lernen im Latent Space

0.1 Motivation

Die regelbasierte Pipeline (Phase 1-4) setzt strukturierte Spalten voraus. ALKIS enthält aber:

- Freitext (Bemerkungen, Lagebeschreibungen, historische Notizen)
- Kategorische Codes in großer Kardinalität (Gemarkungsschlüssel, Nutzungsarten, AAA-Objektarten)
- Geometrien (Polygone, Linien, Punkte) — numerisch nicht direkt vergleichbar
- Cross-Table-Bezüge, die heute nur manuell verlinkt werden (Phase 3)

Ein gelernter Embedding-Raum normalisiert diese Heterogenität: semantisch ähnliche Einträge landen nah beieinander — unabhängig von Tabelle, Typ oder Formulierung.

0.2 Encoder-Zoo (ohne Bilddaten)

Datentyp	Encoder	Output-Dim
Tabellenzeile (mixed)	SCARF / VIME / TabNet (SSL via Masked Column Reconstruction)	128
Freitext (Bemerkung, Lage)	CLIP Text Encoder, multilingual (clip-ViT-B-32-multilingual), Domain-Fine-Tuning auf ALKIS-Glossar	512
Geometrie (Polygon/ Linie/Punkt)	Handcrafted Features (Area, Perimeter, Compactness, Convex-Hull-Ratio, Anzahl Vertices) + kleines MLP (SSL)	64
Kategorische High-Cardinality IDs	Entity Embeddings (skip-gram auf Ko-Occurrence)	32

0.3 SSL-Trainingsziele (kein Label nötig)

- **Masked Column Modeling** (SCARF/VIME): Zufällige Spalten maskieren, rekonstruieren. Lernt Spalten-Abhängigkeiten.
- **Contrastive Learning** (SimCLR-Style): Zwei augmentierte Views derselben Zeile → nah; andere Zeilen → fern. Augmentierung: Column-Dropout, Wert-Jitter, Synonym-Swap bei Text.
- **Cross-Modal Alignment** (CLIP-Prinzip, ohne Bild): (Freitext-Beschreibung, strukturierte Tabellenzeile)-Paare desselben Flurstücks werden aligniert. Ermöglicht Freitext ↔ Tabelle im selben Raum.

0.4 Fusion in einen gemeinsamen Latent Space

Concatenation der modalen Embeddings → Projection Head (MLP, 2 Layer) → **d=64 shared latent**. Anschließend UMAP oder t-SNE auf 2D für Visualisierung und Human-Debug.

Wichtig: die Regeln werden nicht im 2D-Raum gelernt — 2D ist nur erklärbarer Output. Cluster-Logik läuft im 64D-Raum (stabiler).

```
# Embedding-Pipeline pro Zeile (ohne Bild)
z_tab  = scarf_encoder(row_numeric + row_categorical)   # 128
z_text = clip_text(row_freetext_concat)                 # 512
z_geom = geom_mlp(polygon_features(row_geometry))       # 64

z_fused = projection_head(concat([z_tab, z_text, z_geom]))  # 64
z_2d    = umap.transform(z_fused)                           # 2 (viz/debug)
```

Anwendung A — Outlier Detection im Latent Space

A.1 Methodik

- **Density-basiert:** LOF / HDBSCAN im 64D-Raum. Punkte in Low-Density-Regionen = Kandidaten für Datenfehler.
- **Rekonstruktionsfehler:** Autoencoder-Loss pro Zeile. Hoher Loss = unübliche Kombination.
- **Distanz zum Cluster-Zentroid:** z-score im 64D, Ranking der Top-K Ausreißer.

A.2 Erklärbarer 2D-Output

Jeder Ausreißer bekommt (a) 2D-Koordinate im UMAP-Plot, (b) Liste der **nächsten Nachbarn** (die "normal" sind), (c) SHAP-ähnliche Attribution, welche Modalität den Ausreißer treibt (Text? Geometrie? Kategorie-Kombination?).

Kopplung zu Phase 4 (HITL): Ausreißer-Cluster werden dem Domain-Experten als visuelle Karte gezeigt. Dieser markiert: "echter Fehler" / "gültige Ausnahme". Daraus entstehen neue Kandidatenregeln in `validation_rules` (`status=proposed`, `source=ssl_outlier`).

```
import hdbscan, umap
clusterer = hdbscan.HDBSCAN(min_cluster_size=50, prediction_data=True)
labels = clusterer.fit_predict(Z_64d)
outliers = np.where(labels == -1)[0]  # Noise-Punkte = Kandidaten

# 2D für UI
reducer = umap.UMAP(n_components=2, random_state=42)
Z_2d = reducer.fit_transform(Z_64d)
```

Anwendung B — Klassen-Discovery für gezieltes Sampling

Phase 1 der bestehenden Pipeline sampelt 5% zufällig. Problem: seltene, aber wichtige ALKIS-Konstellationen (z.B. "Kirche mit Nebengebäude auf Mischnutzungs-Flurstück") werden unterrepräsentiert → RF/Decision Tree sieht sie nicht → keine Regel.

B.1 Stratified Sampling via Latent Cluster

1. HDBSCAN-Cluster c1..ck im 64D-Raum finden
2. Pro Cluster proportional-zur-Wurzel (oder gleichverteilt) sampeln → seltene Cluster sind garantiert im Trainingssample
3. Dieses stratifizierte Sample ersetzt das Random-Sample in Phase 1/2

B.2 Cluster als Pseudo-Labels

Cluster-IDs können als Zielspalte für Decision Tree genutzt werden — auch wenn die Spalte in den Rohdaten nicht existiert. Der Decision Tree lernt dann: "Welche Attribut-Kombination definiert Cluster 7?" → ergibt Struktur-Regeln, die ohne SSL nie entstanden wären.

```
from sklearn.tree import DecisionTreeClassifier, export_text
dt = DecisionTreeClassifier(max_depth=4, min_samples_leaf=30)
dt.fit(X_original[top_features], cluster_labels)
print(export_text(dt, feature_names=list(top_features)))
# -> SQL CHECK für jeden Cluster-Pfad (source=ssl_cluster)
```

Anwendung C — Cross-Table Feature Linking

Heute (Phase 3) muss ein User semantische Links manuell definieren (z.B. `Flurstück.id → Hausnummer.flurstück_ref`). Der Latent Space schlägt Links automatisch vor.

C.1 Prinzip

Alle Tabellen werden in denselben Latent Space eingebettet (via CLIP-ähnliches Cross-Modal-Training mit bekannten FK-Paaren als Positive Samples). Danach gilt:

> **Hypothese:** Einträge aus Tabelle A in Cluster c_i zeigen mit hoher Wahrscheinlichkeit auf Einträge aus Tabelle B in Cluster c_j (stabiles Mapping c_i^A → c_j^B).

C.2 Link-Discovery Algorithmus

1. Embedde Tabelle A und B separat, aber mit geteiltem Projection Head
2. Cluster beide im gemeinsamen Raum
3. Baue Cross-Cluster-Confusion-Matrix M[i,j] = |{(a,b) ∈ FK: a ∈ c_i^A, b ∈ c_j^B}|
4. Starke Diagonalen / stabile Mappings → Kandidaten für Referenzintegritätsregeln
5. Abweichungen von diesem Mapping → Verdacht auf Datenfehler (verwaiste oder falsch verknüpfte FKs)

C.3 Generierte SQL-Regel (Beispiel)

```
-- SSL-entdeckt: Flurstücke in Cluster 'landwirtschaftlich_groß'
-- referenzieren fast immer Nutzungen in Cluster 'agrar_codes'
-- Abweichung => proposed rule
CHECK (
  NOT (flurstueck_cluster = 'landwirtschaftlich_groß')
  OR nutzung_code IN (SELECT code FROM nutzung WHERE cluster = 'agrar_codes')
)
-- source = ssl_cross_table, confidence = 0.97
```

Anwendung D — Freitext-Validierung via CLIP

ALKIS-Bemerkungsfelder sind heute regelfrei. Mit CLIP-Text-Embeddings können Regeln der Form "Bemerkung passt semantisch zu Nutzungsart" erzeugt werden.

Beispiel-Pattern

- `Nutzungsart = Wohnen` → Bemerkung-Embedding sollte nahe an Cluster "Wohngebäude-Vokabular" liegen. Cosine-Distanz > Threshold → Flag.
- `Gebäudefunktion = Kirche` + Freitext enthält "Lagerhalle" → Embeddings divergieren → Regel: `gebaeude_funktion='Kirche' → freitext_cluster ≠ 'industrial_storage'`

```
# Freitext-Plausibilität
from sentence_transformers import SentenceTransformer
import numpy as np

clip_text = SentenceTransformer('clip-ViT-B-32-multilingual-v1')
emb_freitext = clip_text.encode(df['bemerkung'].tolist())
emb_nutzung  = clip_text.encode(df['nutzung_code_description'].tolist())
cos_sim = (emb_freitext * emb_nutzung).sum(axis=1) / (
    np.linalg.norm(emb_freitext, axis=1) *
    np.linalg.norm(emb_nutzung,  axis=1))
df['freitext_mismatch'] = cos_sim < 0.35  # Kandidaten für Review
```

> **Achtung Threshold:** `0.35` ist ein Platzhalter. In der Praxis muss der Schwellwert *pro Nutzungsart / Gebäudefunktion* kalibriert werden (z.B. per Quantil auf bekannten Positive-Paaren), sonst entstehen massenhaft False Positives.

Phase 1 — Sampling & Profiling

Ausgangspunkt: stratifiziertes Sample (siehe Anwendung B), 2-3 Durchläufe. Ziel: harte Grenzen pro Spalte.

Pro Spalte wird ermittelt:

- **Numerisch:** min, max, mean, stddev, null-rate
- **Kategorisch:** distinct values, null-rate, Häufigkeitsverteilung
- **Strings:** Regex-Muster, Länge min/max

```
col >= min AND col <= max
col IN (cat1, cat2, ...)
col IS NOT NULL
```

Schwellwert `stddev / range < threshold` → Regel wird vorgeschlagen.

Phase 2 — ML-basierte Regelableitung

Schritt 1 — LLM Feature-Vorschläge

LLM erhält Spaltenname + Typ und schlägt vor, welche Spalten voneinander abhängen könnten (z.B. "Gemeindeschlüssel könnte von Kreis abhängen"). *Hinweis: Schema-Grounding nötig, sonst Halluzinationsrisiko.*

Schritt 2 — AutoML Prediction

AutoML (FLAML / AutoSklearn) predictet eine Zielspalte aus allen anderen Spalten.

- Accuracy > Threshold (z.B. 95%) in *beiden* Samples → Regel behalten
- Nur in einem Sample gefunden → Regel verwerfen
- Bei unbalancierten Targets: zusätzlich F1 / Precision pro Klasse prüfen

Schritt 3 — Materialisierung als Code/SQL

Wichtig: Jede Regel wird als SQL oder Python-Code gespeichert — kein ML-Modell als Regel.

```
-- Wenn A -> B
CASE WHEN A IS NOT NULL THEN B IS NOT NULL END

-- Wenn A, B -> C
CHECK (A IS NULL OR B IS NULL OR C = expected_value)
```

Phase 2.5 — Cross-Table SSL-Regeln

Siehe Anwendung C. Output fließt mit `source = ssl_cross_table` in `validation_rules`.

Phase 2.6 — Freitext-Semantik-Regeln

Siehe Anwendung D. Output mit `source = ssl_clip_text`.

Phase 3 — Manuelle Datenverknüpfungen

User definiert semantische Links zwischen Entitäten. Daraus Referenzintegritäts-Regeln als SQL.

- `Flurstück.id → Hausnummer.flurstück_ref`
- `Gebäude.id → Adresse.gebäude_ref`
- `Gemarkung.schlüssel → Flurstück.gemarkung_schlüssel`

```
-- Generierte Regel
SELECT COUNT(*) FROM hausnummer h
LEFT JOIN flurstueck f ON h.flurstueck_ref = f.id
WHERE f.id IS NULL; -- Verletzung: verwaiste Referenz
```

Phase 4 — Human-in-the-Loop

1. Regel vorschlagen (mit Beispiel-Fehlern + korrekten Zeilen + UMAP-Snapshot bei SSL-Herkunft)
2. User bewertet: passt | passt nicht
3. Regel erhält Status: `proposed → active` oder `rejected`
4. Feedback aktualisiert Konfidenz-Score
5. Loop → nächste Regel

> **Gate für SSL-Regeln:** SSL-abgeleitete Regeln (source beginnt mit `ssl_`) erhalten eine höhere Confidence-Schwelle und erfordern zwei Approver, da die Quelle stochastischer ist als Profiling/ML.

Regel-Datenmodell (konsolidiert)

Feld	Typ	Beschreibung
rule_id	UUID	Eindeutige ID
rule_sql	TEXT	SQL oder Python-Code der Regel
status	ENUM	proposed / active / rejected
confidence	FLOAT	0-1
source	ENUM	profiling / ml / manual / ssl_outlier / ssl_cluster / ssl_cross_table / ssl_clip_text
approved_by	TEXT	User, der approved hat
approved_at	TIMESTAMP	Zeitstempel
latent_cluster_id	INT	Zugehöriger HDBSCAN-Cluster (Traceability)

latent_2d_snapshot	URL	Referenz auf UMAP-Plot (on-demand regenerierbar)
embedding_model_version	TEXT	Hash / Tag des SSL-Encoders (Reproduzierbarkeit)

ML-Modelle im Detail

Vorfilter: Random Forest — Feature-Relevanz

RF ist Blackbox, aber `feature_importances_` liefert zuverlässiges Ranking. Nachfolgende interpretierbare Modelle werden auf Top-Features beschränkt.

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

importances = pd.Series(rf.feature_importances_, index=X_train.columns)
top_features = importances[importances > 0.05].sort_values(ascending=False)
# kreis_id       0.62
# gemarkung_key  0.21
# nutzung_code   0.09
# flaeche        0.05
```

1. Decision Trees (Primärmodell)

Jeder Pfad Wurzel→Blatt = IF-THEN. Direkt in SQL übersetzbar. `max_depth=3-5`.

```
from sklearn.tree import DecisionTreeClassifier, export_text
dt = DecisionTreeClassifier(max_depth=4, min_samples_leaf=50)
dt.fit(X_train[top_features.index], y_train)
print(export_text(dt, feature_names=list(top_features.index)))
```

```
-- Regel: kreis_id=12 AND gemarkung_key<=500 -> gemeinde_A
CHECK (
  NOT (kreis_id = '12' AND gemarkung_key <= 500)
  OR gemeinde_id = 'gemeinde_A'
)
```

2. Rule Learners (SkopeRules / RIPPER)

Generiert direkt logische Regelsätze. Kompakter als tiefe DTs. Ignoriert Ausreißer besser.

```
from skrules import SkopeRules
clf = SkopeRules(max_depth=3, precision_min=0.95, recall_min=0.05)
clf.fit(X_top, y_train)
for rule in clf.rules_:
    print(rule[0])
# gemeinde_id == '123' and kreis_id in ['12','14']  (prec=0.99)
# nutzung_code == 'Bauen' and flaeche > 10000        (prec=0.97)
```

```
CHECK (gemeinde_id != '123' OR kreis_id IN ('12','14'))
CHECK (nutzung_code != 'Bauen' OR flaeche <= 10000)
```

3. Association Rules (FP-Growth / Apriori)

Unsupervised. Findet X → Y in kategorischen Daten.

```
from mlxtend.frequent_patterns import fpgrowth, association_rules
from mlxtend.preprocessing import TransactionEncoder

transactions = df[['gebaeude_funktion','dachform','nutzung_code']].values.tolist()
te = TransactionEncoder()
df_enc = pd.DataFrame(te.fit_transform(transactions), columns=te.columns_)
frequent = fpgrowth(df_enc, min_support=0.05, use_colnames=True)
rules = association_rules(frequent, metric="confidence", min_threshold=0.95)
# gebaeude_funktion=Kirche -> dachform=Turm  (conf=0.98)
# nutzung_code=Wohnen      -> etagen <= 5    (conf=0.96)
```

4. Sparse Linear Models (Lasso / GAM)

L1-Regularisierung → wenige nicht-null Koeffizienten → lesbare Formel.

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1).fit(X_num, y_num)
# amtliche_flaeche     1.0
# berechnete_flaeche  -1.0  => Differenz ~ 0
```

```
CHECK (ABS(amtliche_flaeche - berechnete_flaeche) <= 0.5)
```

Modell-Vergleich

Modell	Rolle	Datentyp	Output	Direkt SQL?
Random Forest	Vorfilter	Alle	Feature-Ranking	Nein
Decision Tree	Primär	Num + Kat	IF-THEN-Pfade	Ja
Rule Learner	Primär	Num + Kat	Kompakte Regeln	Ja
Association Rules	Discovery	Kategorisch	X → Y	Ja
Lasso / GAM	Numerisch	Numerisch	Formeln	Ja

Modelle bewusst vermieden

- Random Forest *als Regel* — viele Bäume, nicht lesbar (nur als Vorfilter)
- XGBoost / Gradient Boosting — Blackbox, keine SQL-Übersetzung
- Deep Neural Networks — Millionen Parameter, ungeeignet
- SVM (nichtlinear) — Kernel-Trick, nicht interpretierbar

Tech-Stack (konsolidiert, ohne Bild)

Komponente	Tool	Zweck
Tabulares SSL	SCARF / VIME / PyTorch Tabular	Masked Column Modeling
Text Encoder	sentence-transformers (clip-ViT-B-32-multilingual)	CLIP-Text-Embeddings (DE), Domain-Fine-Tuning
Geometry Features	Shapely + kleines MLP	Polygon-Embeddings
Dim-Reduktion 2D	UMAP, openTSNE	Erklärbare Visualisierung
Clustering	HDBSCAN	Dichte-basiert, Noise = Outlier
Outlier Scoring	PyOD (LOF, IForest, ECOD)	Multi-Method-Konsens
Vector Store	pgvector / FAISS	Nearest-Neighbor-Suche für HITL
Feature Relevanz	scikit-learn RandomForest	feature_importances_-Vorfilter
Decision Trees	scikit-learn	IF-THEN-Regeln
Rule Learner	skrules SkopeRules	Kompakte Regelsätze
Association Rules	mlxtend FP-Growth	Kategorische X → Y
Sparse Linear	scikit-learn Lasso	Numerische Checks
AutoML	FLAML / AutoSklearn	Automatische Modellwahl
Profiling	ydata-profiling / pandas	Phase-1-Statistiken
Regel-Speicher	PostgreSQL	validation_rules-Tabelle
LLM Feature-Vorschläge	GPT / Claude via API	Abhängigkeitsvermutungen
HITL-UI	Streamlit + Plotly (UMAP-Scatter klickbar)	Domain-Experte wählt Punkte direkt

Vorteile der Erweiterung

- **Mehr Regeln aus weniger Daten:** Cluster-basiertes Sampling deckt seltene Fälle ab.
- **Freitext wird validierbar:** erstmals regelbare Semantik über CLIP-Space.
- **Cross-Table-Integrität automatisch:** reduziert manuellen Aufwand in Phase 3.
- **Erklärbarkeit bleibt erhalten:** SSL liefert nur Kandidaten — finale Regel ist weiterhin SQL/Python, vom Experten freigegeben.
- **Visuelle Debug-Schicht:** UMAP-2D-Plot zeigt dem Experten, wo im Datenraum Fehler häufen.

Risiken & Gegenmaßnahmen

Risiko	Gegenmaßnahme

SSL-Embeddings instabil zwischen Trainingsläufen	Seed fixieren, `embedding_model_version` speichern, nur bei Drift neu trainieren
CLIP-Bias auf Englisch / generisches Vokabular	Multilinguales Modell + **pflicht**-Domain-Fine-Tuning auf ALKIS-Glossar
Cluster = Artefakt statt echtes Muster	HITL-Gate: jede SSL-Regel vom Experten bestätigt; zwei Approver bei `source=ssl_*`
2D-UMAP täuscht Struktur vor	Entscheidungen auf 64D-Cluster; 2D nur Viz; Trust-Score anzeigen
CLIP-Threshold willkürlich	Threshold pro Nutzungsart kalibrieren (Quantil auf Positive-Paaren)
Zirkularität bei Cross-Modal Alignment (FKs als Positives)	Nur manuell bestätigte FK-Paare als Positives; restliche FKs sind Testmenge
High-Cardinality kategorische Spalten destabilisieren SCARF	Entity-Embedding-Schritt vorschalten (skip-gram), dann in SCARF einspeisen
Rechenaufwand	Embeddings einmalig berechnen, in pgvector cachen, inkrementell bei neuen Zeilen
Unbalancierte Targets (95%-Accuracy-Threshold unzuverlässig)	F1/Precision pro Klasse zusätzlich; Lift statt reiner Accuracy
Regel-Drift im Produktivbetrieb	Periodisches Re-Evaluieren aktiver Regeln auf Vollpopulation; Alert bei Precision-Drop

Empfohlener Roll-out (Priorisierung)

1. **Phase 1 (Quick Win):** Tabulares SSL + HDBSCAN → stratifiziertes Sampling ersetzt Random-Sample. Geringer Invest, hoher Impact.
2. **Phase 2:** CLIP-Freitext mit Domain-Fine-Tuning + kalibrierte Thresholds pro Nutzungsart.
3. **Phase 3:** Cross-Table-Linking — erst wenn FK-Positive-Pairs sauber kuratiert sind.
4. **Phase 4:** Outlier-Detection als reguläre HITL-Queue für Domain-Experten.

Offene Fragen

- Datenzugang: CSV / Datenbank direkt oder via API?
- Regelformat: SQL bevorzugt oder Python?
- Welche Freitextfelder sind pro Objektart verfügbar und wie befüllt?
- Fine-Tuning-Korpus für CLIP auf ALKIS-Vokabular — existiert ein Domain-Glossar?
- Embedding-Storage: pgvector in bestehender Postgres-Instanz oder separater Vector-DB?
- Re-Training-Kadenz: monatlich / bei Schema-Änderung / eventbasiert?
- Accuracy-/Precision-Schwellen: pauschal 95% oder pro Spalte differenziert?
- Golden-Dataset mit manuell gelabelten Fehlern als Benchmark — vorhanden oder aufzubauen?
- UI: Streamlit ausreichend oder schwereres Framework nötig?